VIETNAM NATIONAL UNIVERSITY - HO CHI MINH CITY
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY

Assignment EW:

Other Indexing Structures and Indexes in Today's DBMSs

Group 2 Presentation

Professor: Dr. Phan Trong Nhan

Ho Chi Minh City, October 2026

Group 02 · Team Members & Contributions

2252396 — **Nguyen Trung Kien**
Indexing theory

2353196 — **Trieu Thuan Toan**
SQL Server benchmarks

2352988 — **Ho Le Minh Quan**
NoSQL indexing theory

2352608 — **Huynh Minh Khoi**
MongoDB benchmarks

2212828 — **Vu Minh Quan**
Coordination & report

2352951 — **Lam Thanh Phuong**
SQL result analysis

2352699 — **Thai Thien Hai Long**
MongoDB result analysis

2352708 — **Dinh Cao Thien Loc**
Dashboard & demo

2352967 — **Ngo Phuong Quang**
Results validation

2352758 — **Phan Minh**
Reproducibility & QA

Presentation Roadmap · Workload First

01

USE CASES

Six realistic query bottlenecks

02

INDEX CHOICE

Match each index to the workload

03

MECHANISMS

How the six index types work

04

EXPERIMENTS

Native DBMS benchmarks and costs

05

DECISIONS

When to use / avoid each index

THREE QUESTIONS THIS STUDY ANSWERS

1. WHAT are the modern index structures and techniques?
2. HOW do SQL Server and MongoDB implement and execute them?
3. WHEN do they improve workloads—and at what cost?

Flow: workloads → hypotheses → native measurements → engineering verdicts

Why “Other Indexing Structures” Matter

STANDARD CLASSROOM CONCEPTS

- Single-column B+ tree
- Dense / sparse index
- Hash-based lookup

BEYOND A BASIC SINGLE-COLUMN INDEX

B-TREE DESIGN

Compound · Filtered/Partial · Covering · Multikey

OTHER PHYSICAL STRUCTURES

Inverted full-text · Compressed columnstore

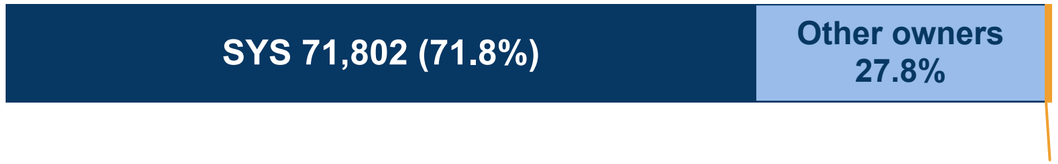
OPTIMIZER ACCESS PATH

Index Seek / Scan · RID Lookup · IXSCAN / FETCH

Creating an index does not guarantee that the optimizer will use it.

UC1: Multi-Column Filtering and Sorting

100,000-row catalog: OWNER distribution



Four query variants

U1a	OWNER + OBJECT_TYPE + OBJECT_ID	2 rows
U1b	OWNER + OBJECT_TYPE (leading prefix)	3,405 rows
U1c	OBJECT_TYPE + OBJECT_ID (missing OWNER)	2 rows
U1d	OWNER = CTXSYS, ORDER BY TYPE, ID	398 rows

SQL Server UC1_Bench_100K

```sql
-- U1a: filter on all three keys
SELECT ID, OWNER, OBJECT_TYPE, OBJECT_ID
FROM dbo.UC1_Bench_100K
WHERE OWNER = N'SYS' AND OBJECT_TYPE = N'TABLE'
  AND OBJECT_ID = 16;
-- U1d: filter by OWNER, sort by the remaining keys
SELECT ID, OWNER, OBJECT_TYPE, OBJECT_ID
FROM dbo.UC1_Bench_100K
WHERE OWNER = N'CTXSYS'
ORDER BY OBJECT_TYPE, OBJECT_ID;
```

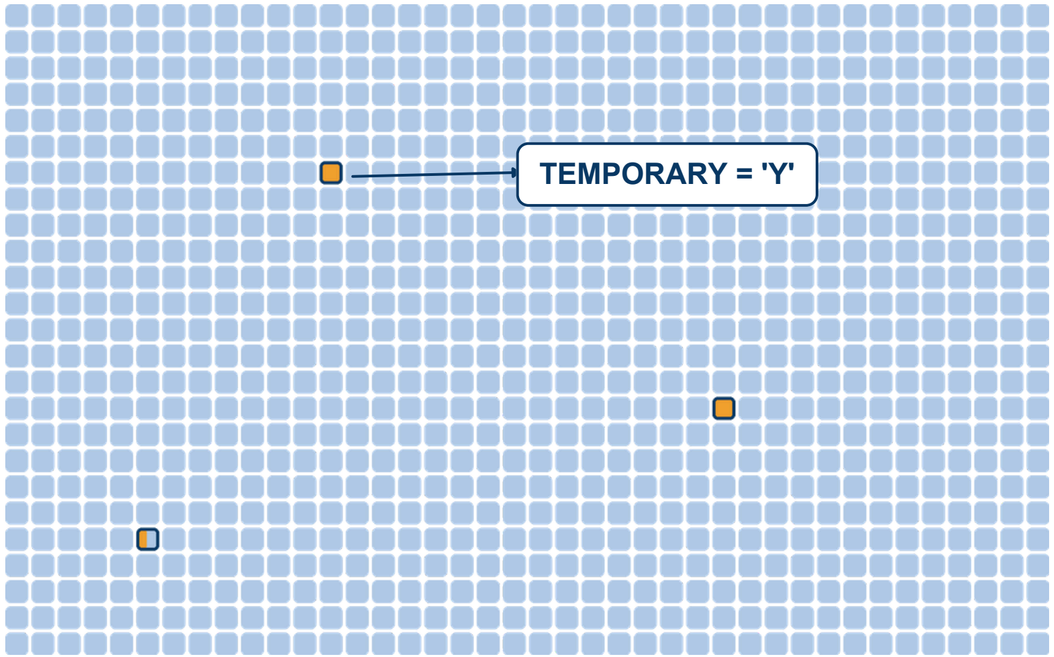
MongoDB uc1_bench_100K

```javascript
// U1a: filter on all three keys
db.uc1_bench_100K.find(
  {OWNER: 'SYS', OBJECT_TYPE: 'TABLE', OBJECT_ID: 16},
  {_id: 0, ID: 1, OWNER: 1, OBJECT_TYPE: 1, OBJECT_ID: 1}
);
// U1d: filter by OWNER, sort by the remaining keys
db.uc1_bench_100K.find(
  {OWNER: 'CTXSYS'},
  {_id: 0, ID: 1, OWNER: 1, OBJECT_TYPE: 1, OBJECT_ID: 1}
).sort({OBJECT_TYPE: 1, OBJECT_ID: 1});
```

Problem: An OWNER-only index reads too many SYS entries and cannot provide the full composite key order. Test full, prefix, missing-key and sorted variants.

UC2: Rare Status Flag (TEMPORARY = 'Y')

1,000,000-row catalog: each cell ≈ 1,000 rows



99.76%
TEMPORARY = 'N'

0.24%
TEMPORARY = 'Y' · 2,405 rows

SQL Server UC2_Bench_1000K

```sql
-- Y: rare subset (the N variant uses N'N')
SELECT OBJECT_TYPE, OBJECT_ID, TEMPORARY
FROM dbo.UC2_Bench_1000K
WHERE TEMPORARY = N'Y' AND OBJECT_TYPE = N'TABLE'
  AND OBJECT_ID BETWEEN 1 AND 10000;
```

MongoDB uc2_bench_1000K

```
// Y: rare subset (N variant: TEMPORARY: 'N')
db.uc2_bench_1000K.find(
  {TEMPORARY: 'Y', OBJECT_TYPE: 'TABLE',
   OBJECT_ID: {$gte: 1, $lte: 10000}},
  {_id: 0, OBJECT_TYPE: 1, OBJECT_ID: 1, TEMPORARY: 1}
);
```

Same OBJECT_ID range at 1M rows; only the flag differs:

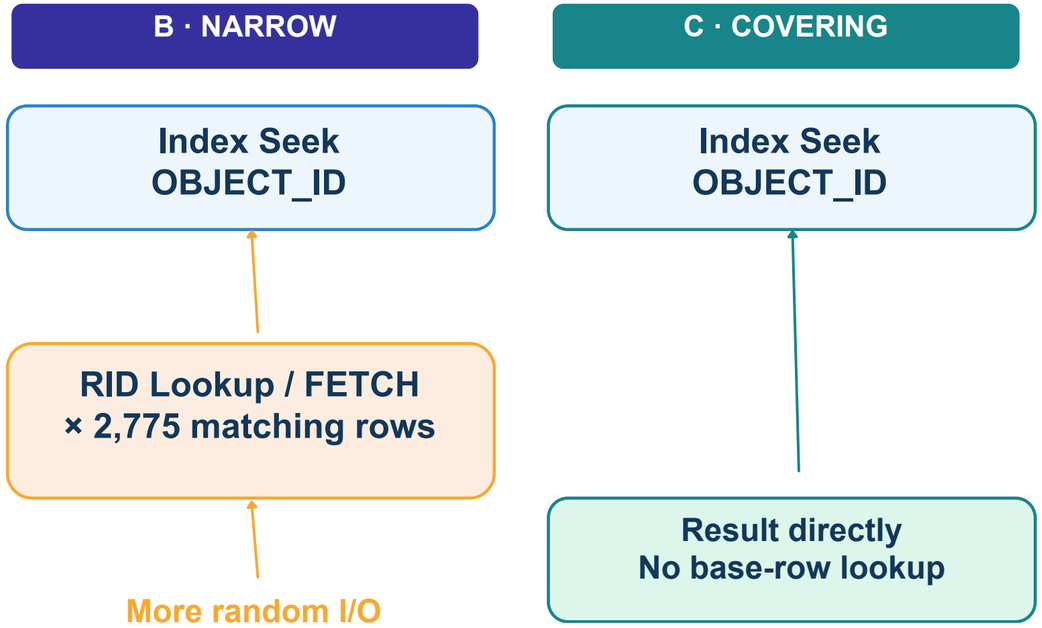
615 rows
matches for 'Y' (dashboard)

15,810 rows
matches for 'N' (batch job)

Problem: Dashboard reads the rare 'Y' subset frequently; occasional 'N' batch jobs need a different access path. A full index stores mostly unused entries.

UC3: Range Queries Returning Multiple Columns

Two access paths for the same range query



Benefit grows with range size; wider index costs more storage.

SQL Server UC3_Bench_1000K

```
-- 1M rows, medium range (2,775 results)
SELECT ID, OWNER, OBJECT_NAME, OBJECT_TYPE
FROM dbo.UC3_Bench_1000K
WHERE OBJECT_ID BETWEEN 1 AND 200;
```

MongoDB uc3_bench_1000K

```
// 1M rows, medium range (2,775 results)
db.uc3_bench_1000K.find(
  {OBJECT_ID: {$gte: 1, $lte: 200}},
  {_id: 0, ID: 1, OWNER: 1,
   OBJECT_NAME: 1, OBJECT_TYPE: 1}
);
```

Three range sizes for the same query
Point: OBJECT_ID = 16 (ID = 1)
Small: 1–50 (100K) or 1–5 (1M)
Medium: 1–1,000 (100K) or 1–200 (1M)

Problem: Thousands of random base-table lookups may cost more than scanning the entire table.

UC4: Natural-Language Search in Documents

10,000 English news articles, BODY field NVARCHAR(MAX)

> …leaders debated **climate change** and its effect on the **stock market**, while the **election** result outside the **white house** moved **bitcoin** prices overnight…

6 queries: 3 single terms (purple), 3 phrases (teal)

election · climate · bitcoin · white house · climate change · stock market

10,000	371 words	918 words
articles (5,000 real + 5,000 fake)	median length (2,213 characters)	p95 length (5,579 characters)

SQL Server A: substring scan · C: full-text

```
-- A: baseline substring scan
SELECT DOC_ID FROM dbo.UC4_Read_A
WHERE BODY LIKE N'%climate change%';
-- C: full-text index search
SELECT DOC_ID FROM dbo.UC4_Read_C
WHERE CONTAINS(BODY, '"climate change"', LANGUAGE 1033);
```

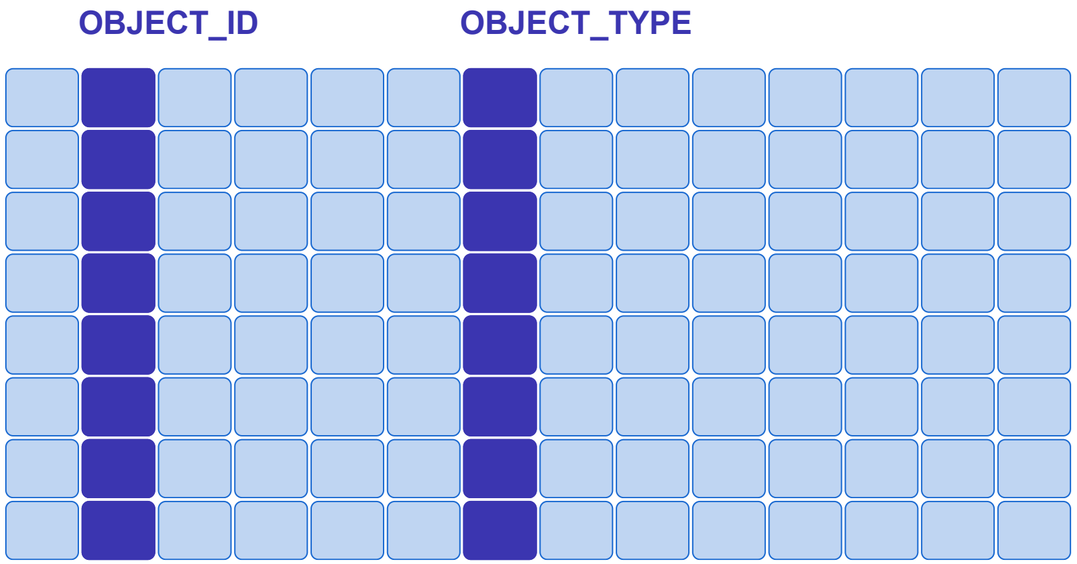
MongoDB A: $regex · C: $text

```
// A: baseline regex scan
db.uc4_read_A.find(
  {BODY: {$regex: 'climate change', $options: 'i'}},
  {_id: 0, DOC_ID: 1}
);
// C: indexed text search
db.uc4_read_C.find(
  {$text: {$search: '"climate change"'}},
  {_id: 0, DOC_ID: 1}
);
```

Problem: Unanchored substring scans are costly. Note: linguistic Full-Text/Text matches are NOT always the same IDs as LIKE / $regex.

UC5: Full-Table Analytical Aggregation

Rowstore: 1,000,000 rows × 14 columns



Purple: 2 columns used by GROUP BY · pale blue: other 12 columns

14 columns
stored together in 8 KB pages

2 columns
OBJECT_TYPE, OBJECT_ID

35 groups
repeated OBJECT_TYPE values

SQL Server UC5_Columnstore

```
-- Aggregation: group 1M rows by OBJECT_TYPE
SELECT OBJECT_TYPE, COUNT_BIG(*) AS n,
       SUM(COALESCE(OBJECT_ID, 0)) AS s
FROM dbo.UC5_Columnstore GROUP BY OBJECT_TYPE;

-- Point probe: ID = 1, no B-tree on ID
SELECT ID, OWNER, OBJECT_NAME, OBJECT_TYPE
FROM dbo.UC5_Columnstore WHERE ID = 1;
```

MongoDB

UC5 runs only on SQL Server: MongoDB has no equivalent clustered columnstore layout in this study.

Problem: Rowstore reads pages containing all 14 columns for a two-column aggregation; columnstore is not a substitute for a selective B-tree point seek.

UC6: Objects with Multiple Tags

Object (PARENT_ID = 127)

owner:SYS | type:TABLE | temporary:Y

```
tags = [ "owner:SYS", "type:TABLE", "temporary:Y" ]
```

4 queries returning PARENT_ID

1 tag	type:TABLE
1 tag	temporary:Y
2 tags	owner:SYS AND type:TABLE
2 tags	type:TABLE AND temporary:Y

10,000	30,471	3.05
parents	tag memberships	tags / parent

SQL Server child table UC6_Read_C_Tag

```
-- Two tags: type:TABLE AND temporary:Y
SELECT PARENT_ID FROM dbo.UC6_Read_C_Tag
WHERE TAG IN (N'type:TABLE', N'temporary:Y')
GROUP BY PARENT_ID HAVING COUNT(DISTINCT TAG) = 2;
```

MongoDB collection uc6_read_C

```
// Two tags: type:TABLE AND temporary:Y
db.uc6_read_C.find(
  {tags: {$all: ['type:TABLE', 'temporary:Y']}},
  {_id: 0, PARENT_ID: 1}
);
```

SQL Server · normalized child table
(127, owner:SYS) (127, type:TABLE) (127, temporary:Y)
MongoDB · embedded array
{PARENT_ID: 127, tags: [owner:SYS, type:TABLE, temporary:Y]}

Problem: Membership lookups by TAG cannot seek on the SQL primary key (PARENT_ID, TAG), so they scan all 30,471 tag rows.

Index Selection Map: From Use Cases to Indexes

UC	Workload signal	Chosen index	Why it fits	Expected weakness
UC1	Multi-column filters + matching ORDER BY	Compound B-tree	Lexicographically ordered keys support prefix seeks and eliminate Sort	Missing leading OWNER key
UC2	Known rare subset (0.24%)	Filtered / Partial	Only qualifying rows are indexed: smaller index and faster build	Queries for 'N'; parameter @p
UC3	Range query projecting many fields	Covering (INCLUDE / compound keys)	All required fields are in the index; no base-table lookups	Tiny ranges; much larger index
UC4	Word and phrase search in prose	Full-Text / Text (inverted)	Token posting lists replace character-by-character scans	Substring semantics; write overhead
UC5	Full-table aggregate over few columns	Clustered Columnstore	Read only needed compressed column segments	Small writes; point lookups
UC6	Multi-valued attributes (tag arrays)	Multikey / child index (TAG, PARENT_ID)	Each tag creates an index entry for direct tag seeking	Two-tag queries can still FETCH

Each index comes with a hypothesis about when it will fail. The results section tests both benefits and weaknesses.

Compound B-tree Index · Ordered Composite Keys

LEXICOGRAPHIC KEY ORDER

CTXSYS	INDEX	19
CTXSYS	TABLE	12
SYS	INDEX	8
SYS	TABLE	16

LEADING PREFIX

OWNER = CTXSYS
contiguous entries

FULL-KEY SEEK

SYS + TABLE + 16
one narrow lookup

MISSING OWNER

TYPE / ID scattered
across owner ranges

Physical structure: B-tree. Technique: key ordering and left-prefix matching.

What it is

Multiple ordered keys in one B-tree index.

SQL Server

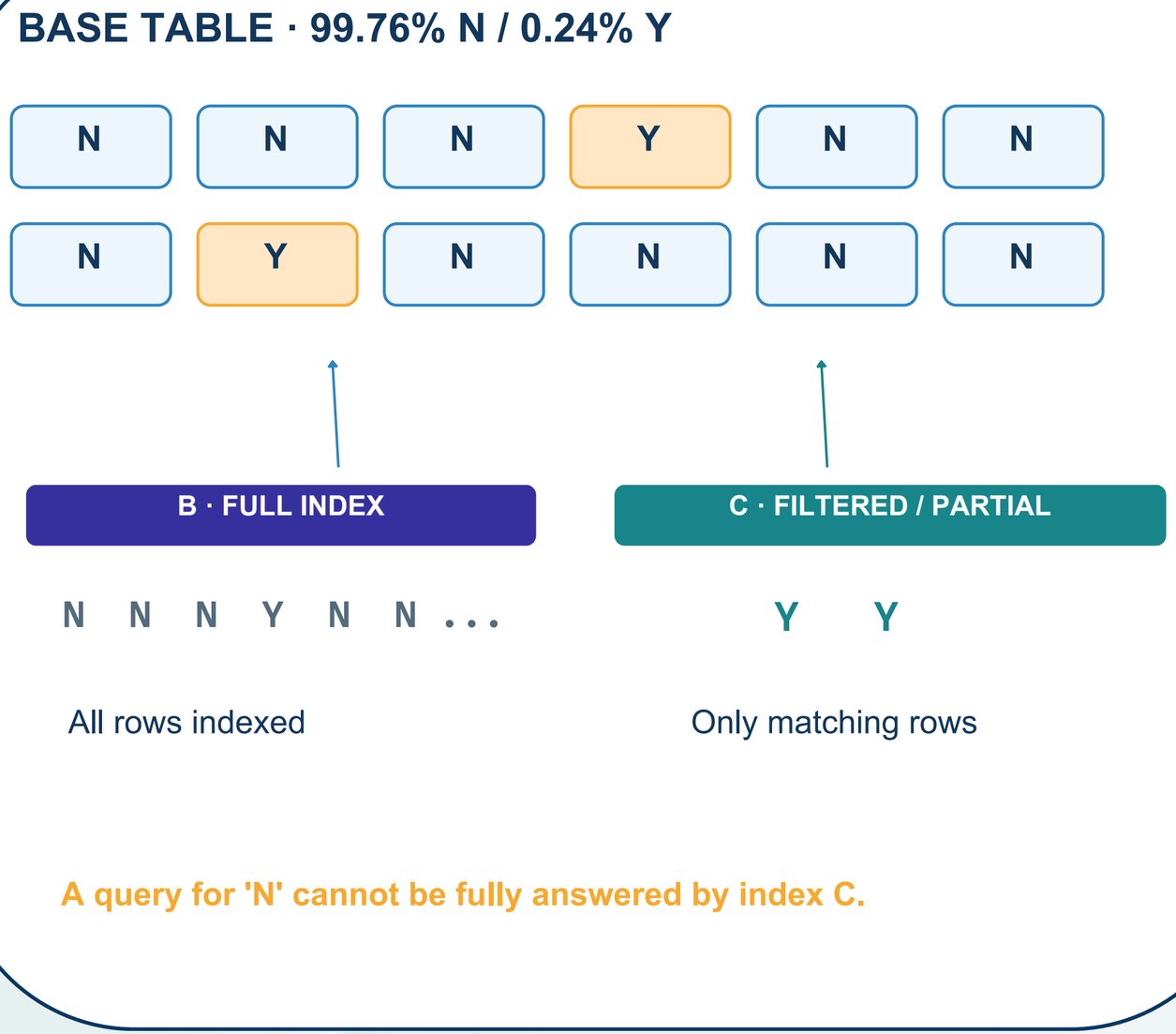
```
CREATE INDEX ix ON T
(OWNER, OBJECT_TYPE, OBJECT_ID);
```

MongoDB

```
db.T.createIndex({OWNER:1,
 OBJECT_TYPE:1, OBJECT_ID:1})
```

Benefit: precise seeks and possible Sort elimination.
Trade-off: larger keys and write maintenance.

Filtered / Partial Index · Index the Rare Subset



What it is

A B-tree containing only rows satisfying a fixed predicate.

SQL Server

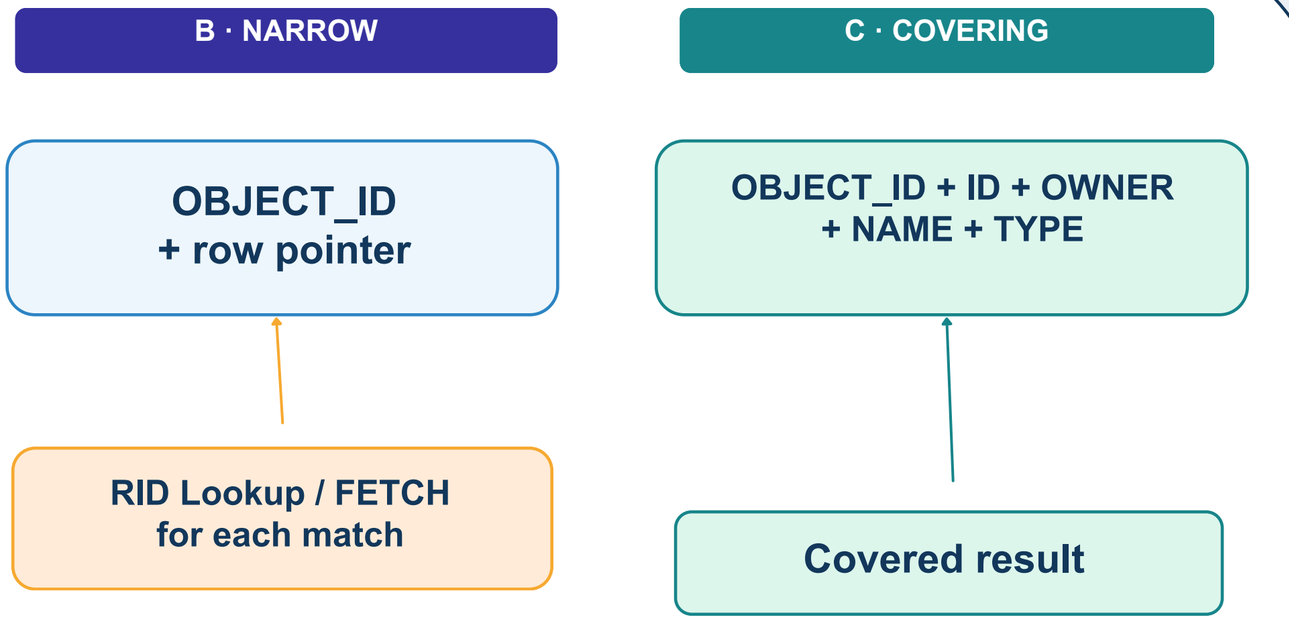
```
CREATE INDEX ix ON T
(OBJECT_TYPE, OBJECT_ID)
WHERE TEMPORARY = N'Y';
```

MongoDB

```
db.T.createIndex({OBJECT_TYPE:1,
 OBJECT_ID:1, TEMPORARY:1},
 {partialFilterExpression:{TEMPORARY:'Y'}})
```

Benefit: tiny index, fast build.
Trade-off: only eligible predicates can use it.

Covering Index · Eliminate Base-Row Lookups



B: index points back to source rows.
C: requested fields are already in the index.

What it is

An index that contains all fields needed by the query.

SQL Server · leaf INCLUDE

```
CREATE INDEX ix ON T(OBJECT_ID)
INCLUDE (ID, OWNER,
 OBJECT_NAME, OBJECT_TYPE);
```

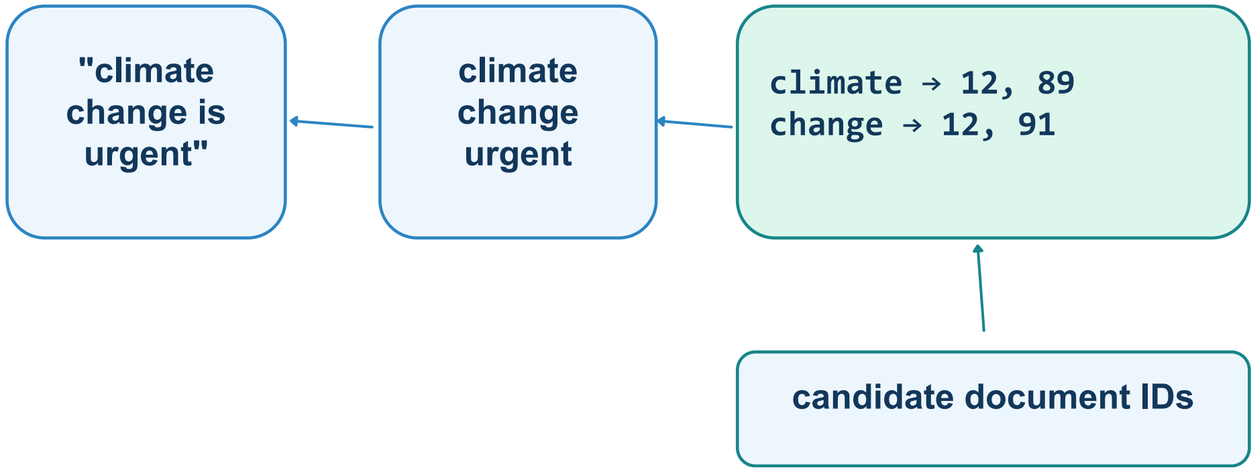
MongoDB · compound keys

```
db.T.createIndex({OBJECT_ID:1,
 ID:1, OWNER:1, OBJECT_NAME:1,
 OBJECT_TYPE:1})  // exclude _id
```

Benefit: no RID Lookup / FETCH.
Trade-off: much wider index.

Inverted Index · Full-Text and Text Search

ARTICLE → TOKENS → POSTING LISTS → IDs



Phrase search must verify adjacency; stems, word boundaries and stop words affect matching.

Physical structure

An inverted mapping from searchable tokens to document postings.

SQL Server · Full-Text

```
CREATE FULLTEXT INDEX ON T
(BODY LANGUAGE 1033)
KEY INDEX PK_T;
```

MongoDB · Text

```
db.T.createIndex({BODY:'text'},
 {default_language:'english'})
```

Benefit: fast word/phrase retrieval.
Costs: changed semantics, write overhead; SQL freshness lag.

Clustered Columnstore · Compressed Column Segments

B · ROWSTORE

```
TYPE | ID | OWNER | ...
TYPE | ID | OWNER | ...
TYPE | ID | OWNER | ...
TYPE | ID | OWNER | ...
```

C · COLUMNSTORE

```
TYPE   TABLE | VIEW | ...
ID     16 | 17 | ...
OWNER  SYS | SYS | ...
```

GROUP BY TYPE; SUM(ID)

The engine reads compressed TYPE and ID segments, rather than every row payload.

Physical structure · SQL Server only

A clustered storage format organized by compressed per-column segments.

SQL Server

```
CREATE CLUSTERED COLUMNSTORE
INDEX CCI_T ON dbo.T;
```

Mechanism

Rowgroups up to 1,048,576 rows; compressed segments and batch processing.

Trade-off: small INSERTs use delta rowgroups; UPDATEs may mark deleted rows.

Multikey vs. Normalized Child Index · Tag Membership

MONGODB · ARRAY

parent 127
tags: [owner:SYS, type:TABLE, temporary:Y]

```
owner:SYS → 127
type:TABLE → 127
temporary:Y → 127
```

SQL SERVER · CHILD TABLE

```
PK(PARENT_ID, TAG)
(127, owner:SYS)
(127, temporary:Y)
```

INDEX(TAG, PARENT_ID)
Tag-first seeks

Two-tag MongoDB $all: seek a selective tag → FETCH candidates → test the other tag.

How it works

MongoDB automatically indexes each distinct array element; SQL indexes normalized tag rows.

SQL Server

```
CREATE INDEX ix ON
UC6_Tag(TAG, PARENT_ID);
```

MongoDB

```
db.uc6_read_C.createIndex(
  {tags:1})
```

Benefit: selective tag seeks.
Costs: multiple entries per parent, FETCH for two-tag checks.

Experimental Datasets · Three Workload-Specific Corpora

01 CATALOG METADATA

10K / 100K / 1M rows

14 fields · UC1, UC2, UC3, UC5

OWNER: SYS 71.8% | CTXSYS 0.4%
TEMPORARY=Y: 0.24%

02 NEWS ARTICLES

10,000 articles

5,000 real + 5,000 fake · UC4

BODY length: median 371 words

95th percentile: 918 words

03 OBJECT TAGS

10,000 parents

30,471 memberships · UC6

3.05 tags per parent

3 to 5 tags per object

Purpose: measure how data distribution changes index usefulness.

The distribution matters: frequent keys, rare flags, long text, and arrays lead to different index choices.

Index Configurations · What A, B, and C Mean

A · Baseline
No optional index

B · Conventional
Common / narrow index

C · Target
Workload-tailored index

UC	A · BASELINE	B · CONVENTIONAL	C · TARGET
UC1	No optional index	Index on OWNER	Compound 3-key B-tree
UC2	No optional index	Full composite index	Filtered / partial Y-only
UC3	No optional index	Narrow OBJECT_ID	Covering (+ output cols)
UC4	Substring scan	— Not applicable —	Full-text / text index
UC5	— Not used —	Rowstore heap	Clustered columnstore
UC6	No tag index	— Not applicable —	Multikey / tag-first child

Compare A/B/C only within the same DBMS; dashes mean that a configuration does not apply.

Evaluation Metrics · Speed, Work, Plans, and Cost

01 READ LATENCY

How fast is the query?

p50 median · p95 tail latency
Speedup = baseline p50 / target p50

Run-time timer, 25 measured repetitions

02 PHYSICAL WORK

How much data was touched?

SQL: 8 KB logical page reads
MongoDB: keys & docs examined

STATISTICS IO / explain(executionStats)

03 EXECUTION PATH

Why did the plan change?

SQL: Seek, Scan, Sort, RID Lookup
Mongo: IXSCAN, FETCH, COLLSCAN

Actual execution plan / stage tree

04 INDEX COST

What do we pay for speed?

Size · Build time · INSERT/UPDATE
UC4: new-token search visibility

Catalog/DMV stats, indexSizes, write tests

Correctness gate: compare row counts or exact result IDs before claiming a same-result speedup.

Benchmark Protocol · Reliable and Reproducible

1 — PREPARE
Clone isolated read/write copies

2 — WARM UP
2 untimed executions

3 — MEASURE
25 timed executions

4 — VERIFY
Count / IDs match expected

5 — DIAGNOSE
Capture plans & index costs

FOUR FAIRNESS RULES

✓ Separate read and write copies
Writes must not modify the measured read dataset.

✓ No plan capture during timing
Execution-statistics collection adds measurement overhead.

✓ Compare within one engine
A/B/C within SQL or within Mongo; no cross-engine ranking.

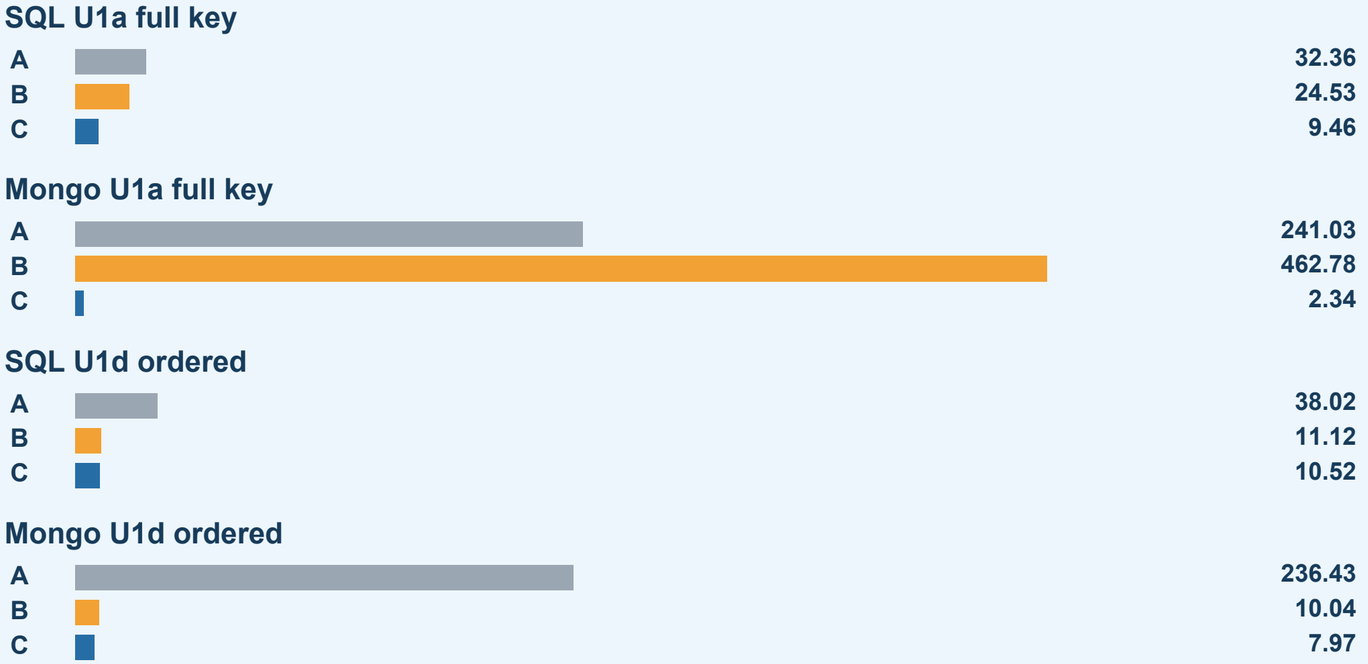
✓ Check output correctness
Same row count or IDs; watch UC4 semantic differences.

Measured latency excludes index build time and diagnostic plan collection.

UC1 Results · Compound B-tree

100K catalog · multi-column filters and ORDER BY

Median read latency (ms) · A/B/C within each DBMS



WHY IT HAPPENED

Aligned keys enable narrow seeks; U1d eliminates Sort.
Missing OWNER prevents a leading-key seek.

COST / COUNTEREXAMPLE

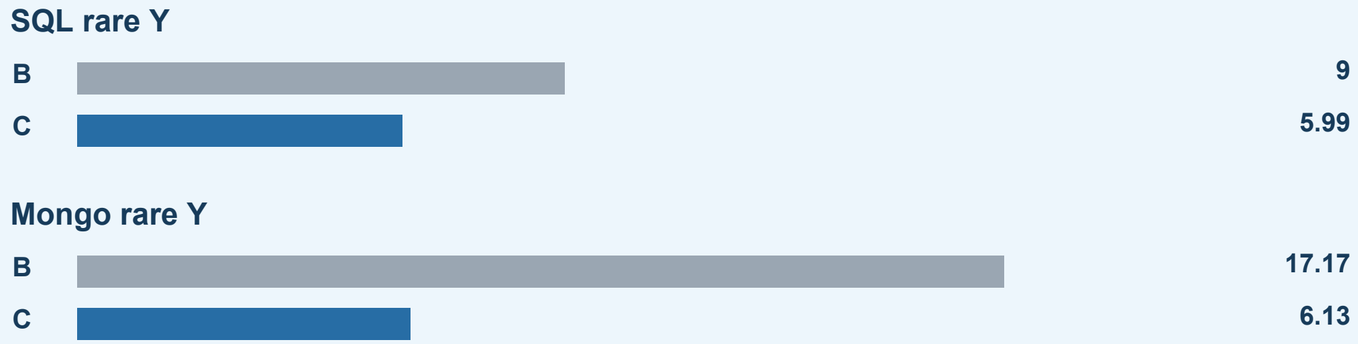
Index C vs B storage: SQL +96.5%; Mongo +87.8%.
Mongo INSERT throughput: −33.5% vs A.

CONDITIONAL · Good for aligned predicates and sort order.

UC2 Results · Filtered / Partial Index

1M catalog · TEMPORARY = Y is only 0.24% of rows

Rare-subset query, median latency (ms) · B full vs C filtered



WHY IT HAPPENED

C stores only 2,405 Y rows instead of 1 million.
N queries cannot use the Y-only index.

COST / COUNTEREXAMPLE

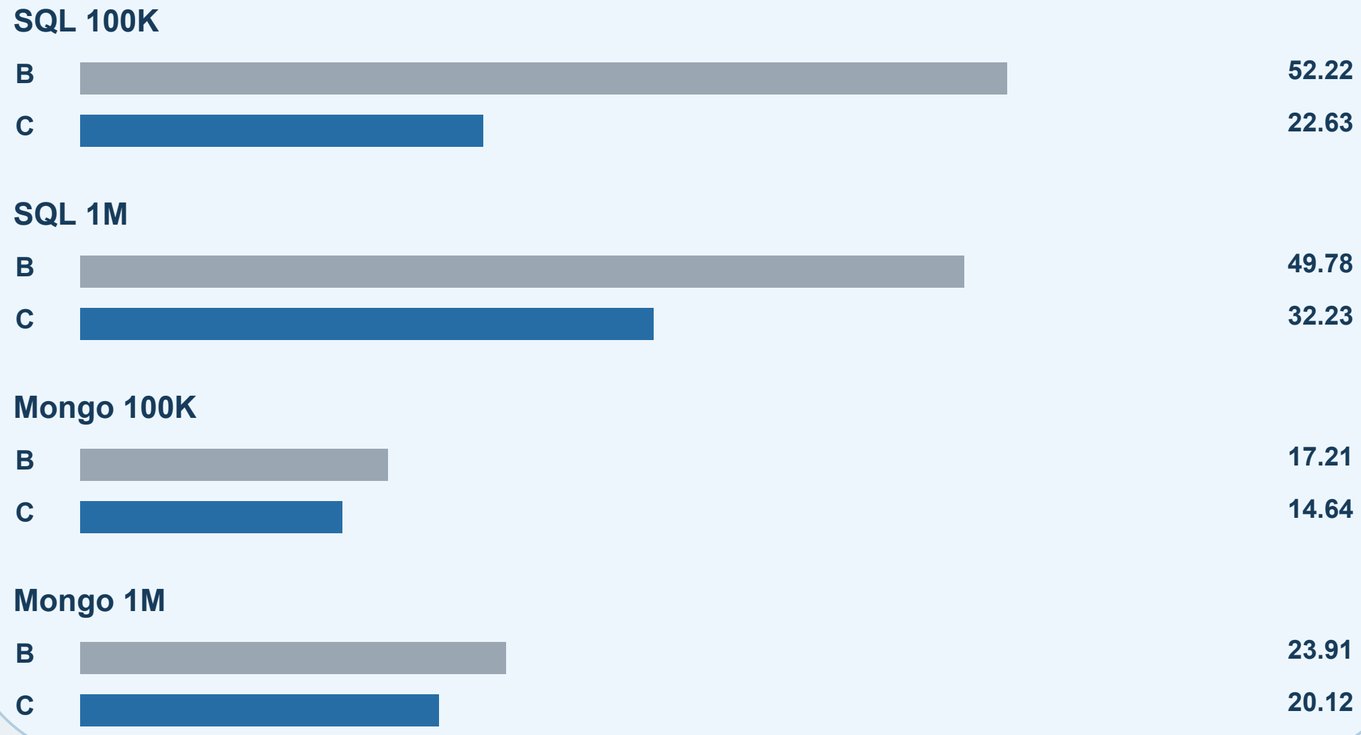
SQL B 43.74 → C 0.13 MiB (−99.71%).
Mongo B 11.95 → C 0.05 MiB (−99.57%).

GOOD · For stable, frequently queried rare subsets.

UC3 Results · Covering Index

100K / 1M catalog · range returns 4 other columns

Medium-range median latency (ms) · B narrow vs C covering



WHY IT HAPPENED

Covering C removes every RID Lookup / FETCH in plans.
Wider keys can lose on small, cached ranges.

COST / COUNTEREXAMPLE

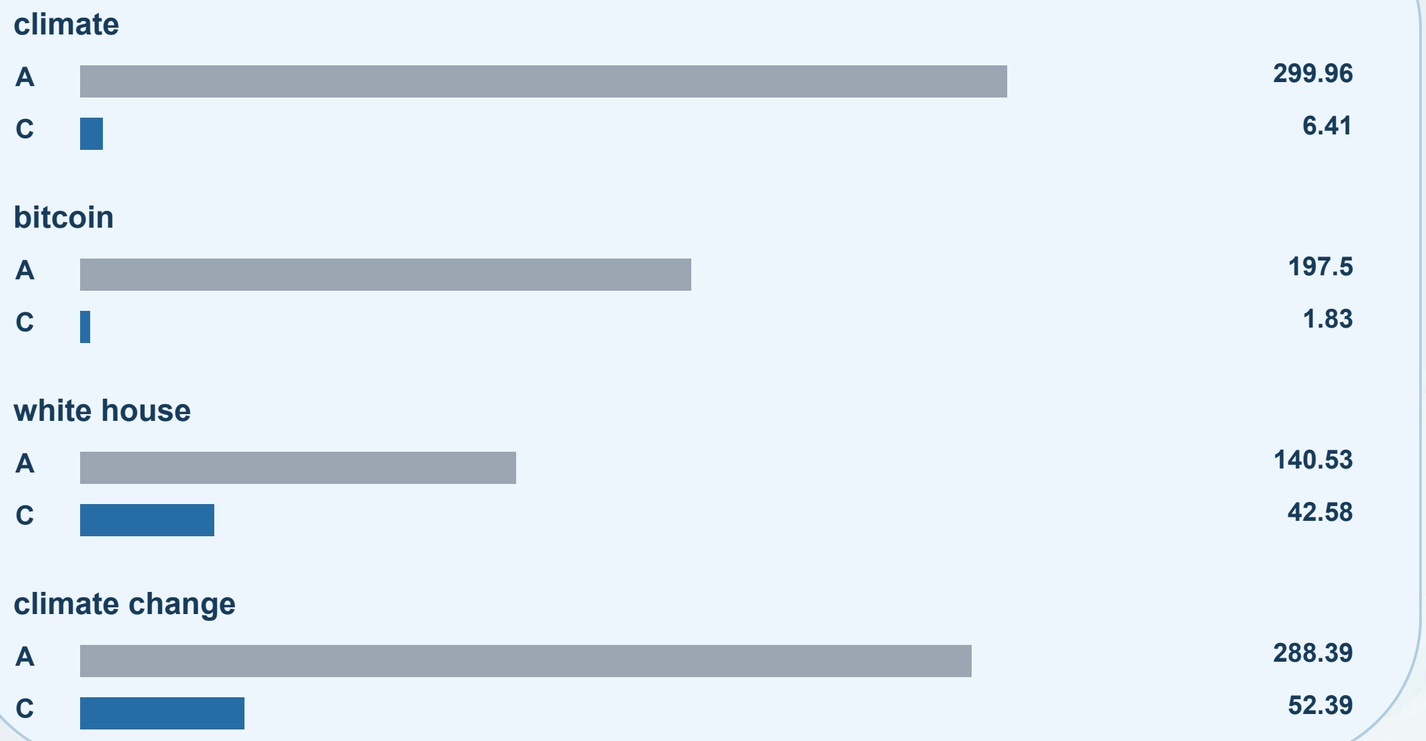
1M index size C/B: SQL 5.94×; Mongo 10.47×.
Mongo 100K small: 2.83 → 4.40 ms (regression).

CONDITIONAL · Favor frequent medium-range projections.

UC4 Results · Full-Text / Text

10K English articles · word and phrase searches

Mongo same-result examples, median latency (ms): A regex vs C text



WHY IT HAPPENED

Inverted token postings avoid character-by-character scans.
Substring vs linguistic search can return different IDs.

COST / COUNTEREXAMPLE

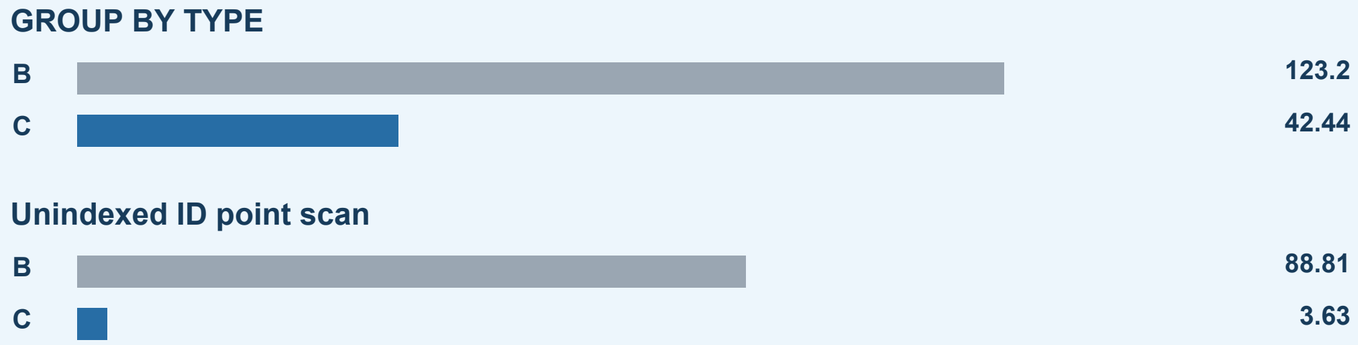
SQL full-text INSERT −26.5%; visibility lag 5.8 s.
Mongo text INSERT −95.3%; UPDATE 38.71 → 115.47 ms.

GOOD for linguistic search · NOT an exact substring replacement.

UC5 Results · Clustered Columnstore

1M catalog · SQL Server only · 35 GROUP BY groups

SQL Server read median latency (ms) · B rowstore vs C columnstore



WHY IT HAPPENED

Aggregation reads compressed column segments.
Both aggregate plans used batch mode; not a mode switch.

COST / COUNTEREXAMPLE

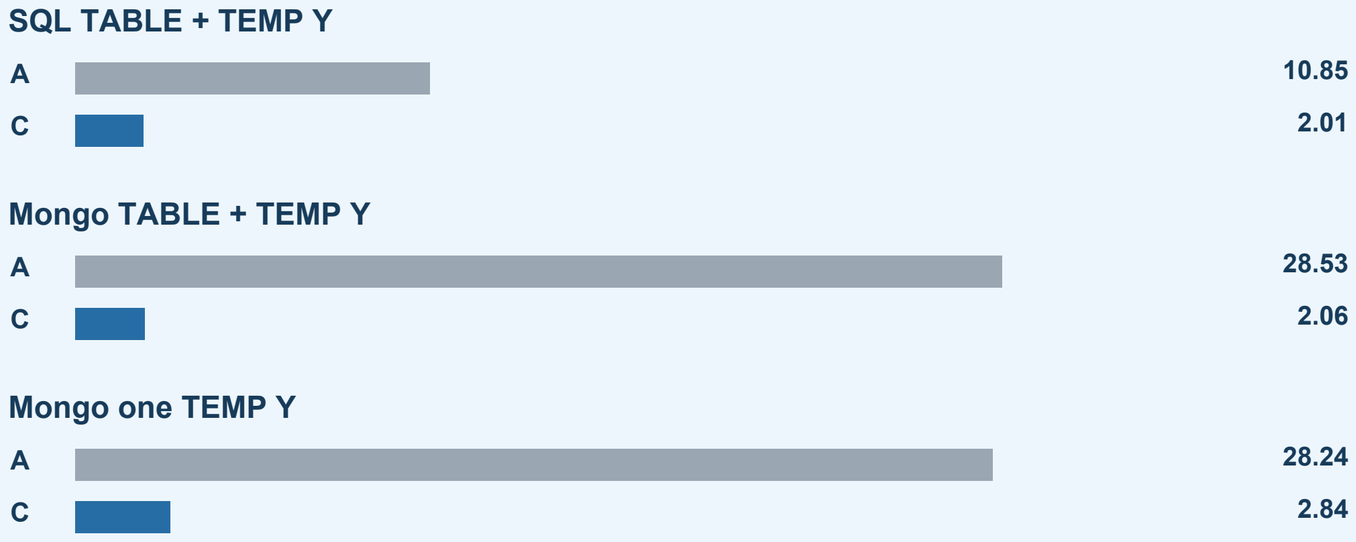
Table storage 219.00 → 9.46 MiB (23.15:1).
Small INSERT throughput: 13,265 → 6,642 rows/s.

GOOD for analytics · Point probe still scans (no ID seek).

UC6 Results · Multikey / Child Index

10K parents · 30,471 tags · 3.05 per parent

Two-tag selective query, median latency (ms) · A vs C



WHY IT HAPPENED

Tag-first B-tree can seek directly to selective tags.
Mongo $all still FETCHes 51 candidates for 41 matches.

COST / COUNTEREXAMPLE

Mongo multikey INSERT throughput −29.3%.
Common-tag queries gain only ~1.4–1.7×.

GOOD when a queried tag is selective.

Conclusion · Did the Chosen Index Solve the Workload?

EVIDENCE-BASED VERDICT: RIGHT INDEX ≠ ALWAYS FASTER

UC / TARGET	WHERE IT WORKS	WHERE IT FAILS OR COSTS	VERDICT
UC1 Compound	Aligned filter / sort; Mongo U1a ~103×	Missing leading key; skewed OWNER can regress	**Conditional**
UC2 Filtered	Y-only: >99.5% storage saved	N queries / unsafe SQL parameters	**Good**
UC3 Covering	Range reads; SQL 100K 2.31×	Small ranges; index grows 6–10×	**Conditional**
UC4 Full-text	Word and phrase searches	Different match semantics; write overhead	**Qualified**
UC5 Columnstore	SQL aggregates 2.90×; 23.15:1 compression	Small INSERTs ~50% slower	**Good**
UC6 Multikey	Selective tags; Mongo two-tag 13.85×	Common tags; candidate FETCH	**Good**

Takeaway: optimize for a measured workload, and assess read gains together with semantics, storage and writes.

Engineering Takeaways · How to Choose an Index

START WITH THE QUERY / DATA DISTRIBUTION

Multi-field WHERE / ORDER BY	→	**Compound B-tree**
Known rare predicate	→	**Filtered / Partial**
Range + extra output columns	→	**Covering**
Words / phrases in text	→	**Inverted Full-Text**
Scan & aggregate few columns	→	**Columnstore**
Membership in array tags	→	**Multikey / tag-first child**

WHAT WE LEARNED

1 Selectivity and key order matter more than “having an index.”

2 An index may reduce I/O but enlarge storage and slow writes.

3 Different query semantics cannot justify same-result speedups.

LIMITATIONS

Single workstation · warm cache · 25 timed runs
No concurrent-client or cold-cache evaluation

Final principle: inspect the actual execution plan, verify result equivalence, then measure total index cost. · Q&A

THANK YOU !!